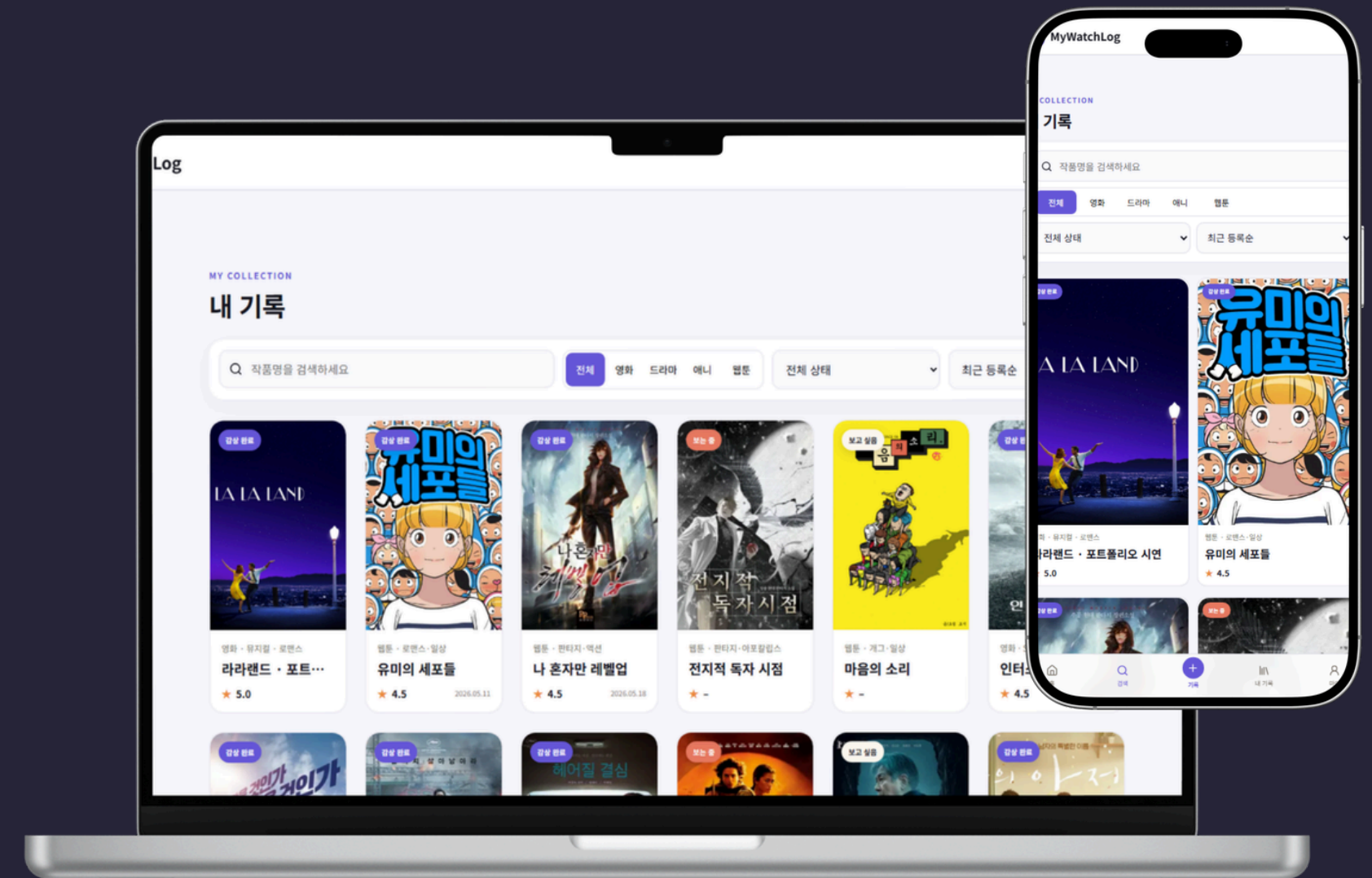


PERSONAL PROJECT · WEB APPLICATION

# MY WATCH LOG

Movie & Drama Personal Watch Archive

보고, 남기고, 다시 꺼내보는  
개인 감상 기록 서비스



# 프로젝트 소개

영화·드라마·애니·웹툰을 보고 난 뒤 별점과 감상평을 기록하고, 나중에 쉽게 다시 찾아볼 수 있는 개인 감상 기록 웹 서비스

## WHY

### 기록이 여기저기 흩어짐

OTT 시청 기록과 개인 감상을 따로 관리해야 해, 예전에 본 작품과 당시 생각을 다시 찾기 어려움

## WHAT

### 한곳에서 기록

작품 유형, 시청 상태, 별점, 감상평, 메모와 날짜를 하나의 기록으로 관리

## VALUE

### 원하는 기록을 빠르게 찾기

색·필터·정렬 기능을 통해 예전에 본 작품과 나의 감상 기록을 쉽게 다시 확인

핵심 사용자 가치

개인 기록 관리 · 빠른 검색 · PC/모바일 사용

## 문제 정의 & 해결 방향

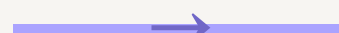
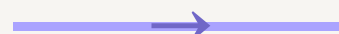
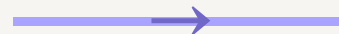
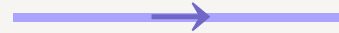
### PROBLEM

시청 기록이 여러 곳에 흩어짐

감상평과 메모를 별도로 관리

예전에 본 작품을 찾기 어려움

다른 사용자의 기록과 구분 필요



### SOLUTION

모든 감상 기록을 한곳에서 등록·조회·수정·삭제

별점·상태·감상평·메모를 하나의 기록으로 관리

검색·유형·상태·정렬 필터 제공

로그인한 사용자별로 기록을 분리하여 저장

## 프로젝트 목표

- |    |             |                                    |
|----|-------------|------------------------------------|
| 01 | 개인별 기록 관리   | 로그인한 사용자마다 자신의 기록만 확인하고 관리         |
| 02 | 기본 기록 기능 완성 | 작품 등록·조회·수정·삭제 기능 구현               |
| 03 | 빠른 기록 검색    | 검색과 필터·정렬을 이용해 원하는 작품을 쉽게 탐색       |
| 04 | 반응형 경험      | 화면 크기에 맞게 레이아웃과 메뉴가 변경되는 반응형 UI 구현 |

# 개발 환경 & 기술 스택

## FRONTEND

React ·  
TypeScript · Vite

화면 구성, 페이지 이동,  
사용자 입력 및 반응형  
UI 구현

## BACKEND

Java 17 · Spring  
Boot 3

회원 및 콘텐츠 REST API  
와 서버 비즈니스 로직 구현

## DATA & AUTH

MySQL · JPA  
JWT · Spring  
Security

회원과 감상 기록 저장 및  
사용자별 데이터 조회

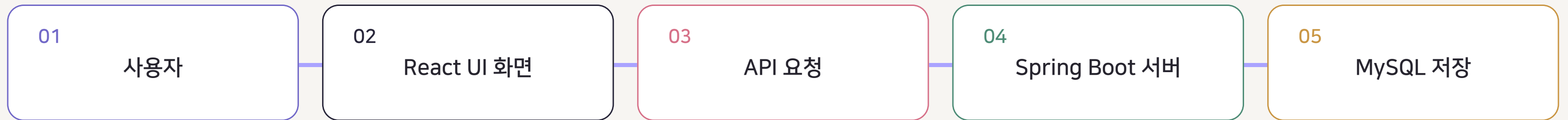
로그인 인증과 사용자별  
데이터 접근 제어

## TOOLS

Git · Docker  
Compose · AI

버전 관리, 로컬 DB 환경  
구성, AI 기반 코드 구현·  
수정·검증

# 시스템 아키텍처



로그인 후 발급받은 JWT를 API 요청에 함께 전달하고,  
서버에서 사용자를 확인한 뒤 해당 사용자의 데이터만 조회·저장

## 프로젝트 디렉터리 구조

### MyWatchLog/

#### ├─ frontend/

| ├─ src/App.tsx 화면·라우트·상태

| ├─ src/api.ts Axios 인증 경계

| └─ styles.css 반응형 레이아웃

#### ├─ backend/

| └─ src/main/java/com/mywatchlog/

| ├─ web/ REST · DTO · 예외

| ├─ security/ JWT · Security

| ├─ domain/ User · Content

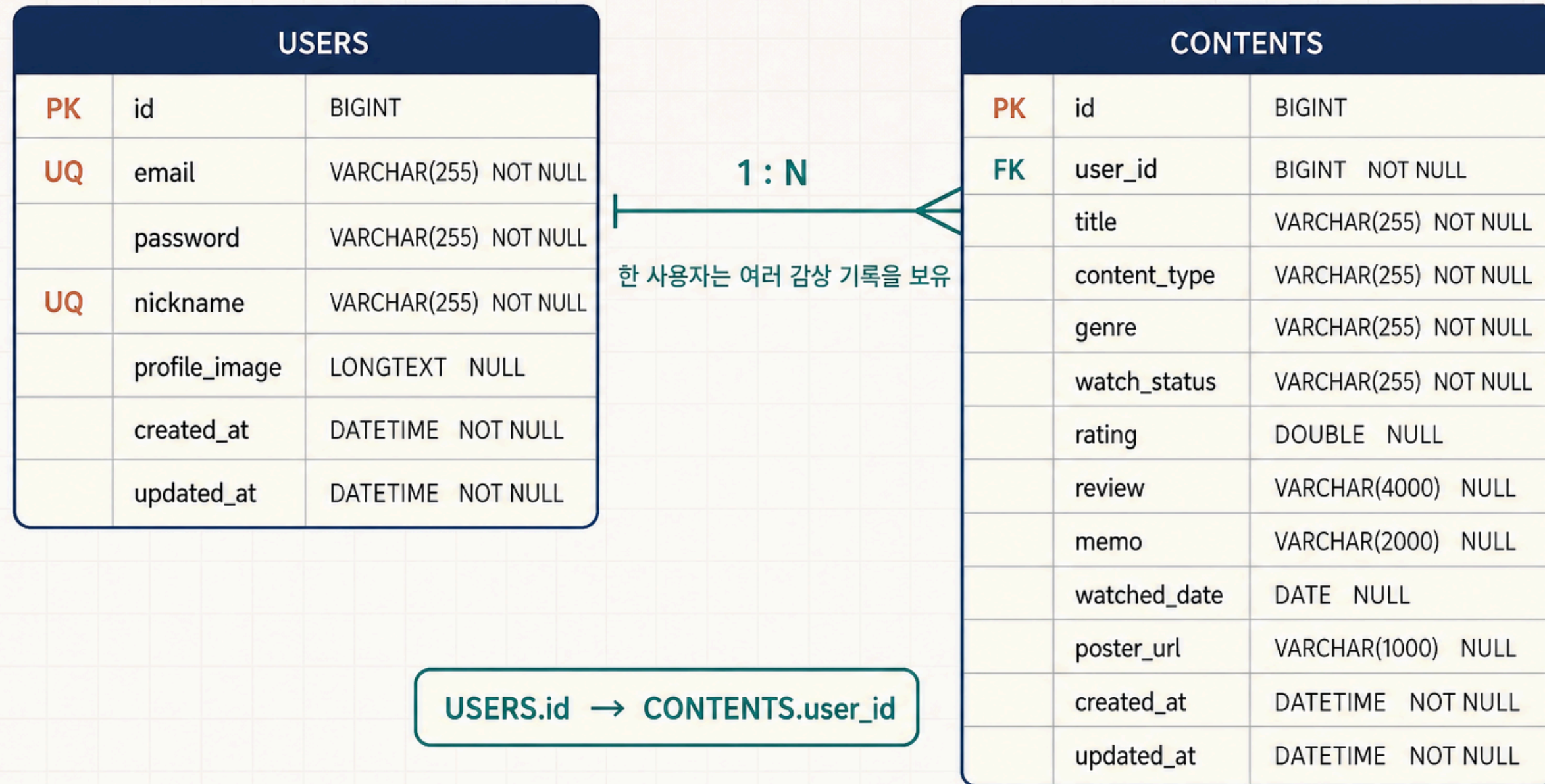
| └─ repository/ JPA 조회

└─ docker-compose.yml MySQL

# 데이터 구조 / ERD

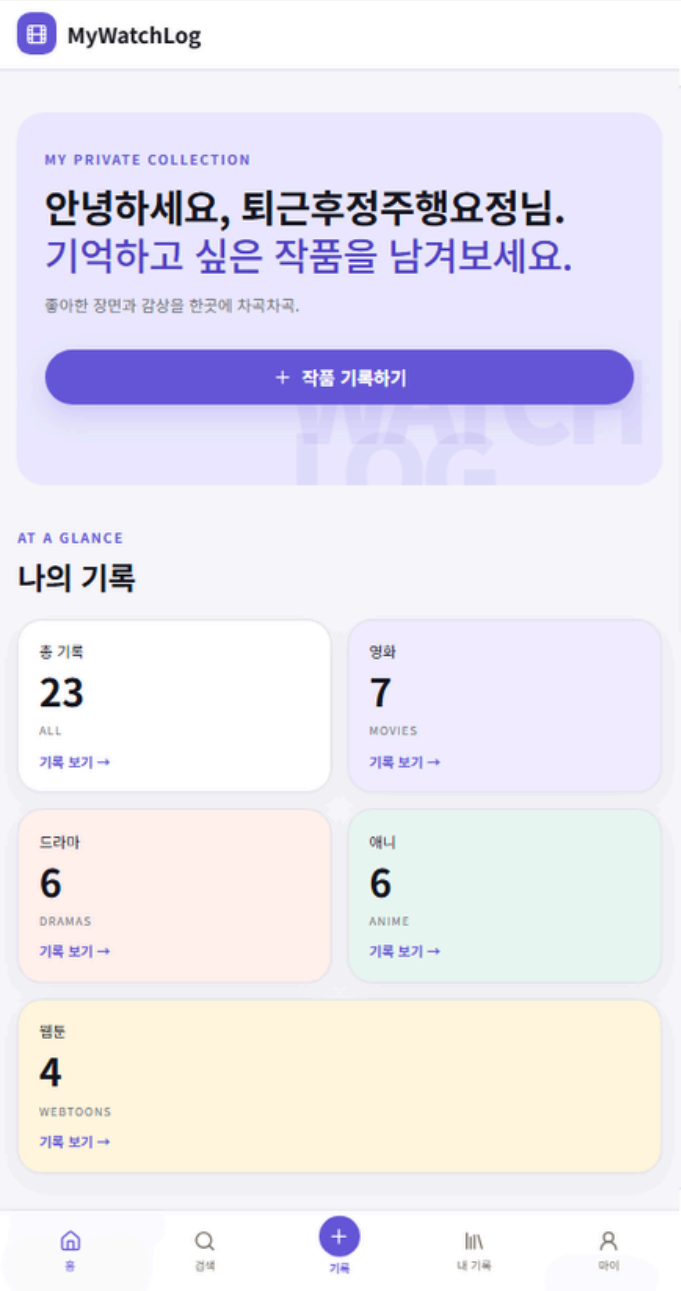
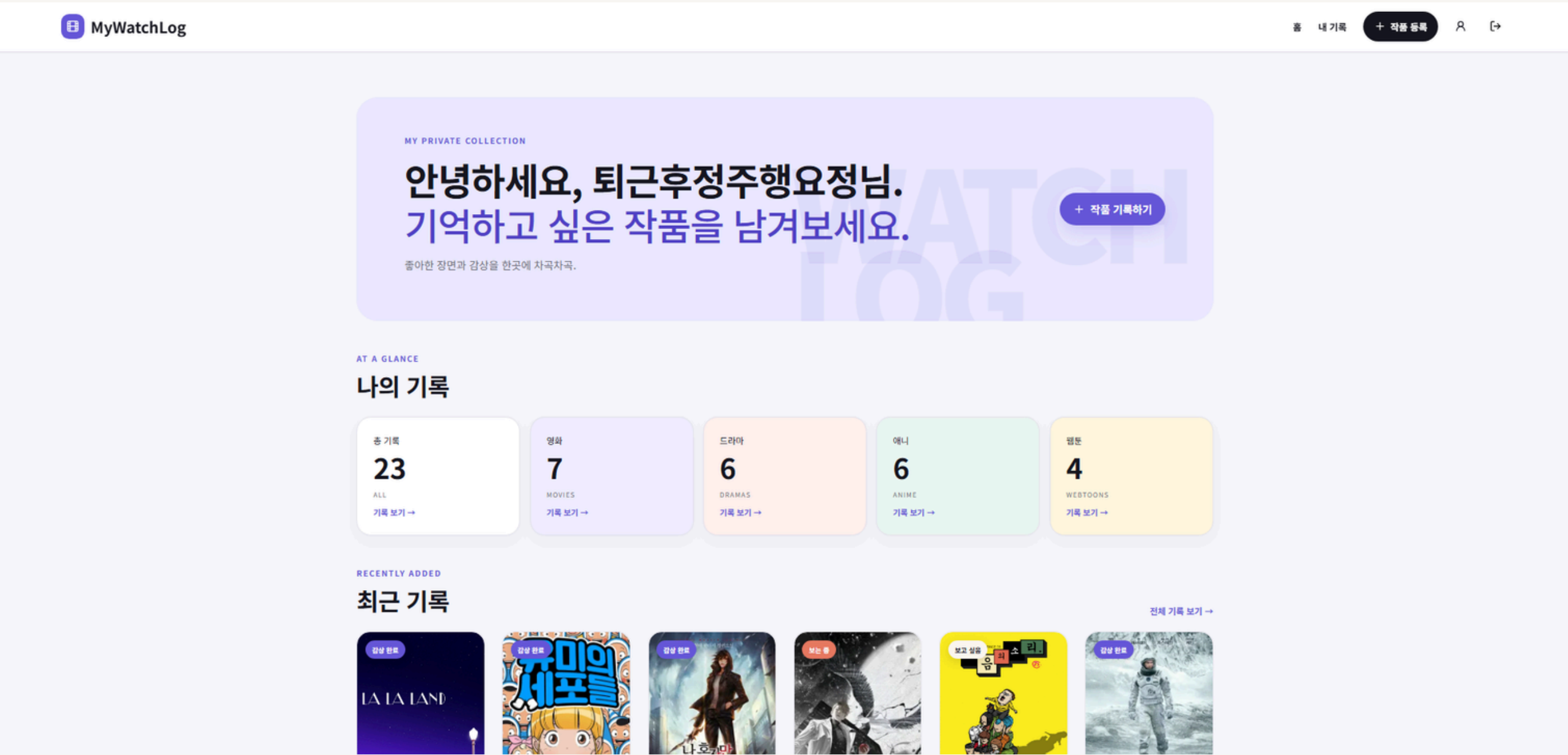
## MyWatchLog ERD

MySQL · Spring Data JPA



content\_type: MOVIE · DRAMA · ANIME · WEBTOON  
 watch\_status: WANT\_TO\_WATCH · WATCHING · COMPLETED

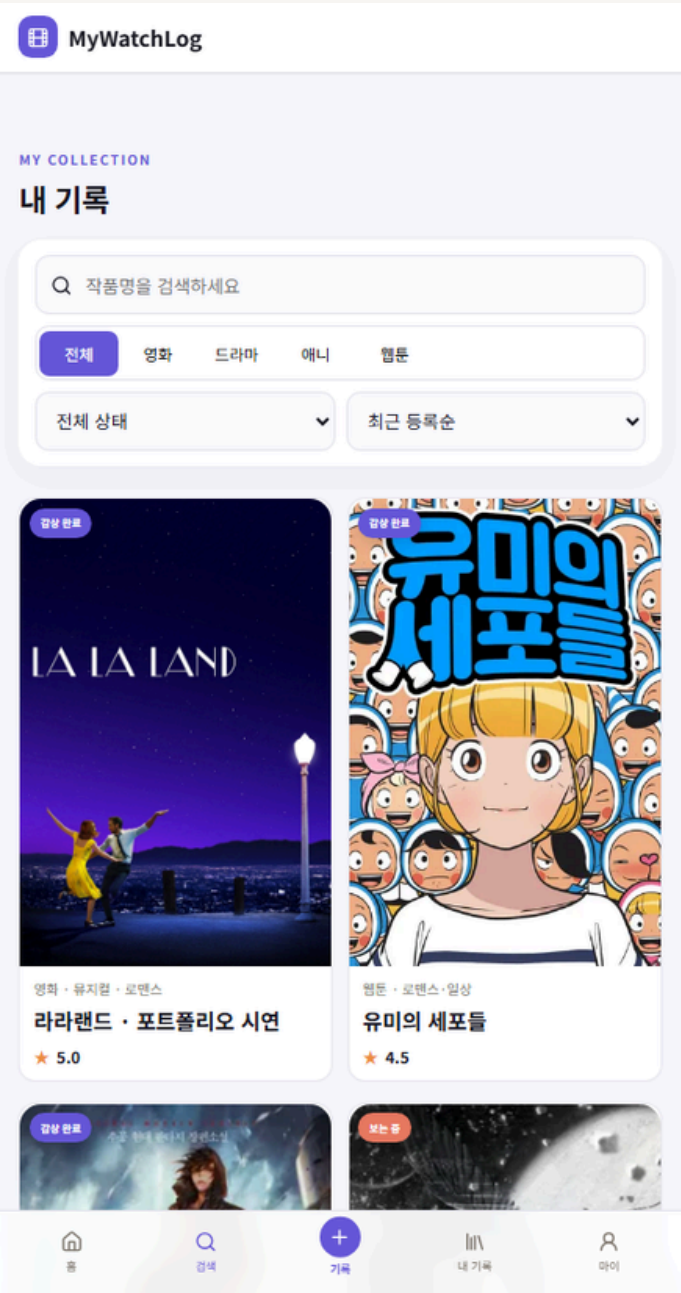
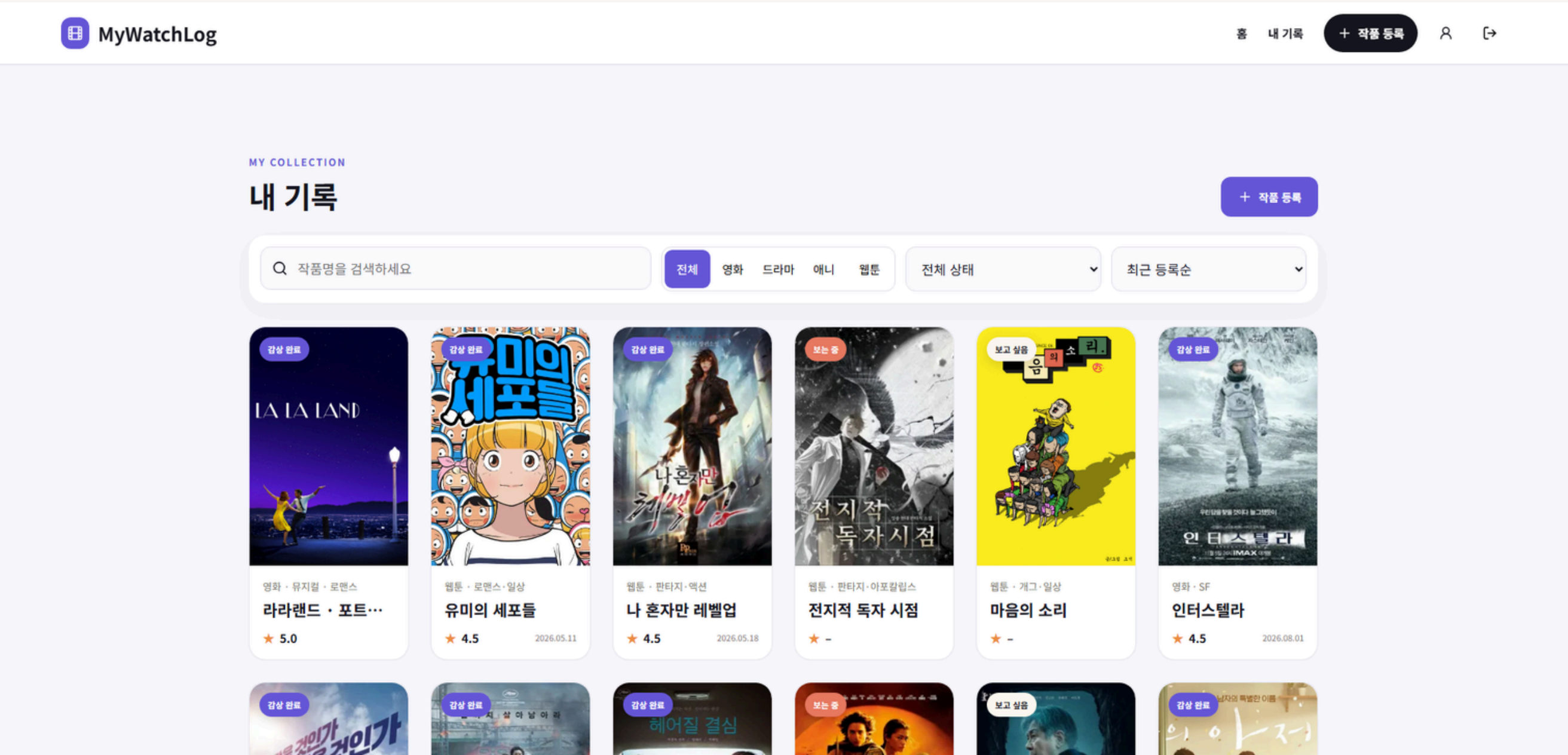
하



기록 수와 최근 기록을 첫 화면에서 확인할 수 있도록 구성했음

PC / MOBILE

# 전체 기록



저장한 작품을 카드 형태로 확인하고, 제목 검색과 작품 유형·감상 상태·정렬 조건을 조합해 원하는 기록을 빠르게 찾을 수 있도록 구현

MOBILE

MyWatchLog

← 돌아가기

NEW RECORD

새 작품 기록

이 작품과 함께한 순간을 기록해보세요.

작품 제목 \*

작품명을 입력하세요

유형 \*

영화

장르 \*

예: SF, 로맨스

감상 상태 \*

보고 싶음

별점

선택 안 함

감상일

연도·월·일

포스터 URL

https://...

나의 감상평

어떤 점이 기억에 남았나요?

홈

검색

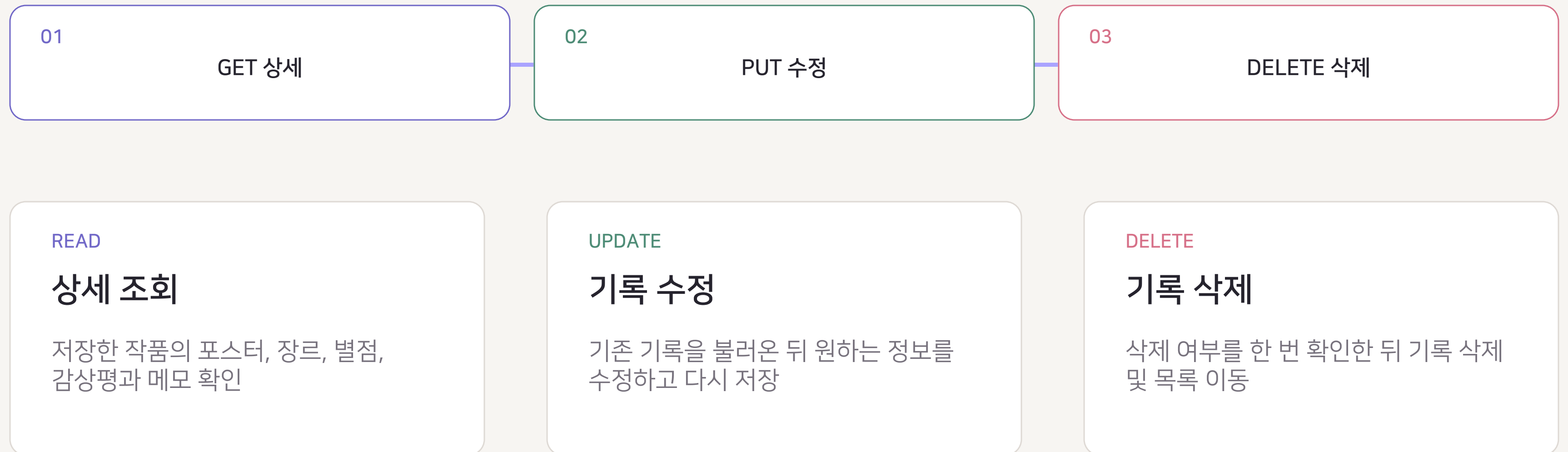
기록

내 기록

마이

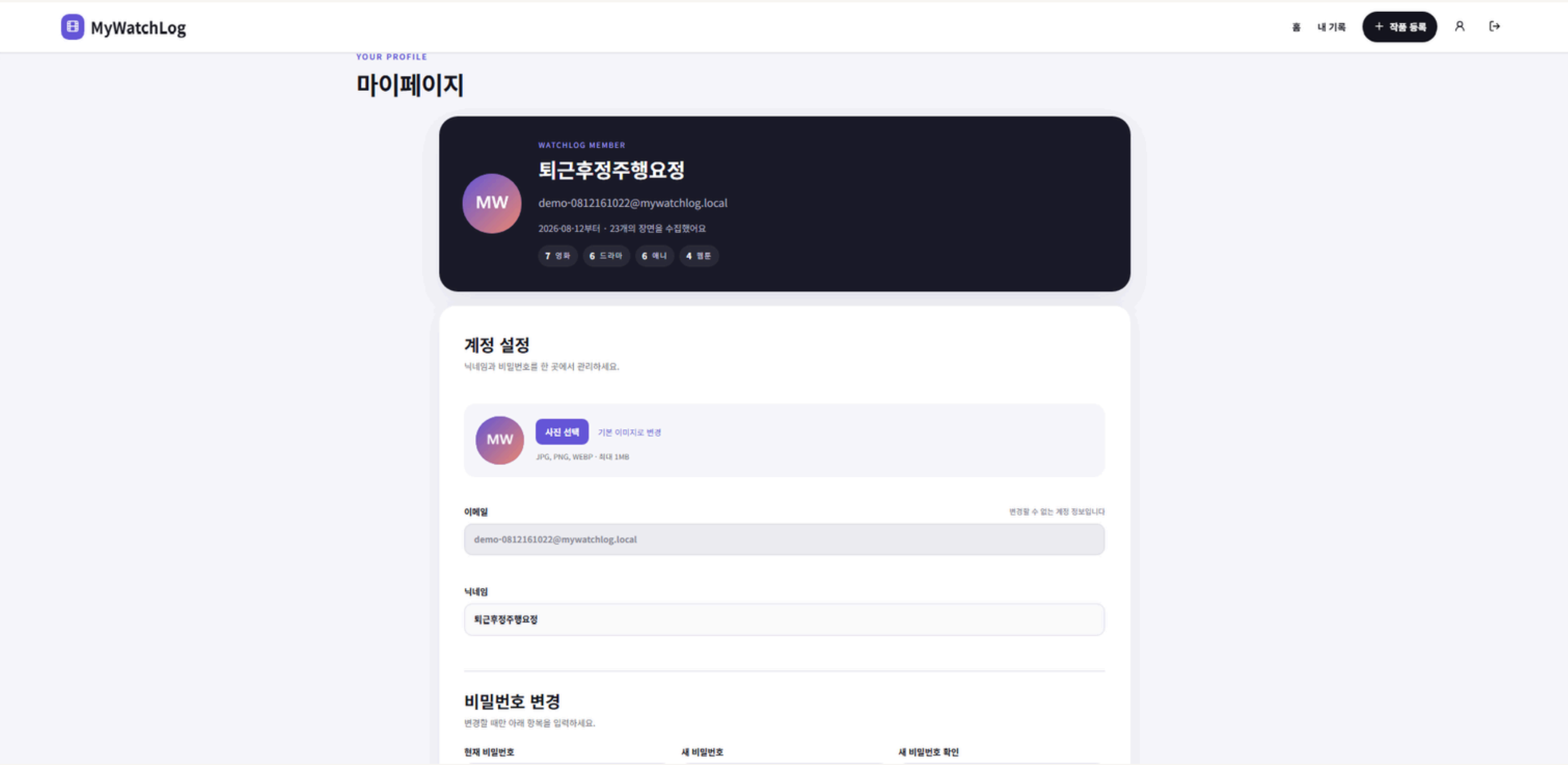
MOBILE

## 기록 상세 / 수정

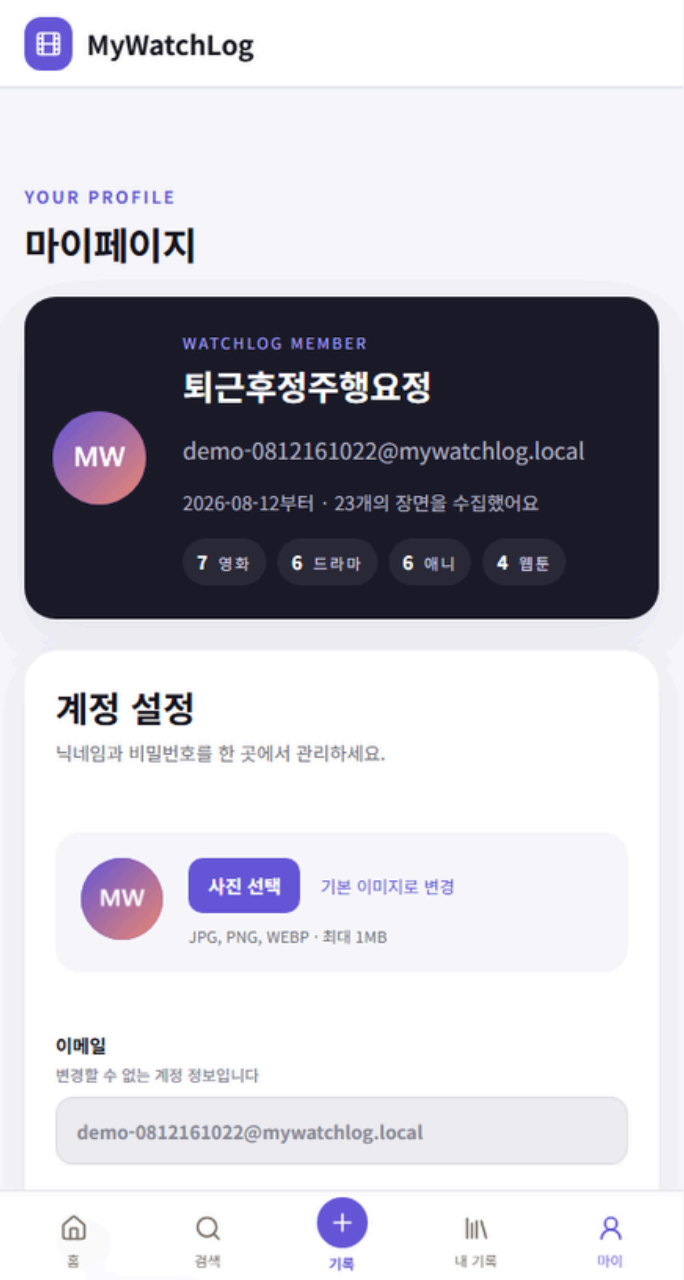


요청된 기록이 현재 로그인한 사용자의 기록인지 서버에서 다시 확인하도록 구현

# 마이페이지

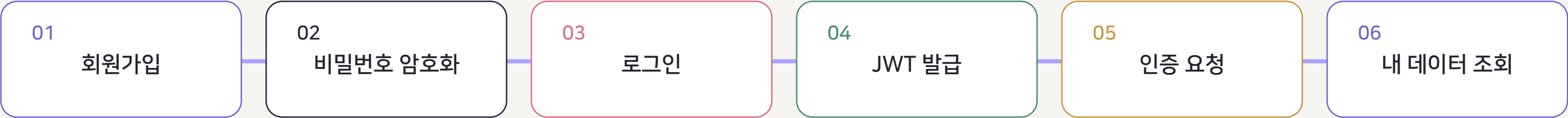


프로필 이미지와 닉네임을 수정하고 계정 정보와 비밀번호를 관리할 수 있는 개인 설정 화면을 구현



MOBILE

# 로그인 / 회원가입



Frontend      로그인 성공 시 JWT를 저장하고 이후 API 요청에 자동으로 전달

Security      전달받은 JWT가 유효한지 확인하고 로그인 사용자를 식별

Controller      현재 로그인한 사용자 정보를 기준으로 데이터를 조회·저장

Routing      로그인하지 않은 사용자가 보호된 페이지에 접근하면 로그인 화면으로 이동

## 핵심 개발 포인트

01

### 사용자별 기록 분리

로그인 계정 기준으로 자신의 기록만  
접근

02

### 검색·필터·정렬 조합

여러 검색 조건을 한 API에서 함께 처리

03

### 클라이언트 + 서버 이중 검증

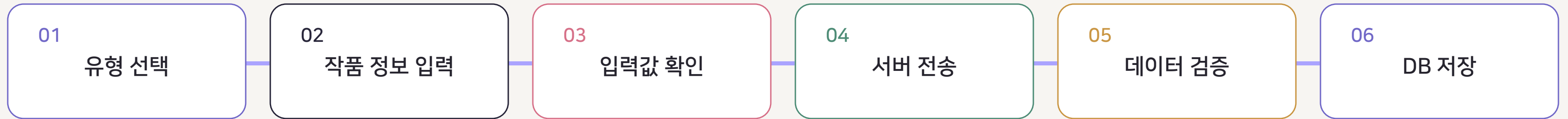
잘못된 데이터가 저장되지 않도록  
두 단계 검증

04

### PC·모바일 반응형 UI

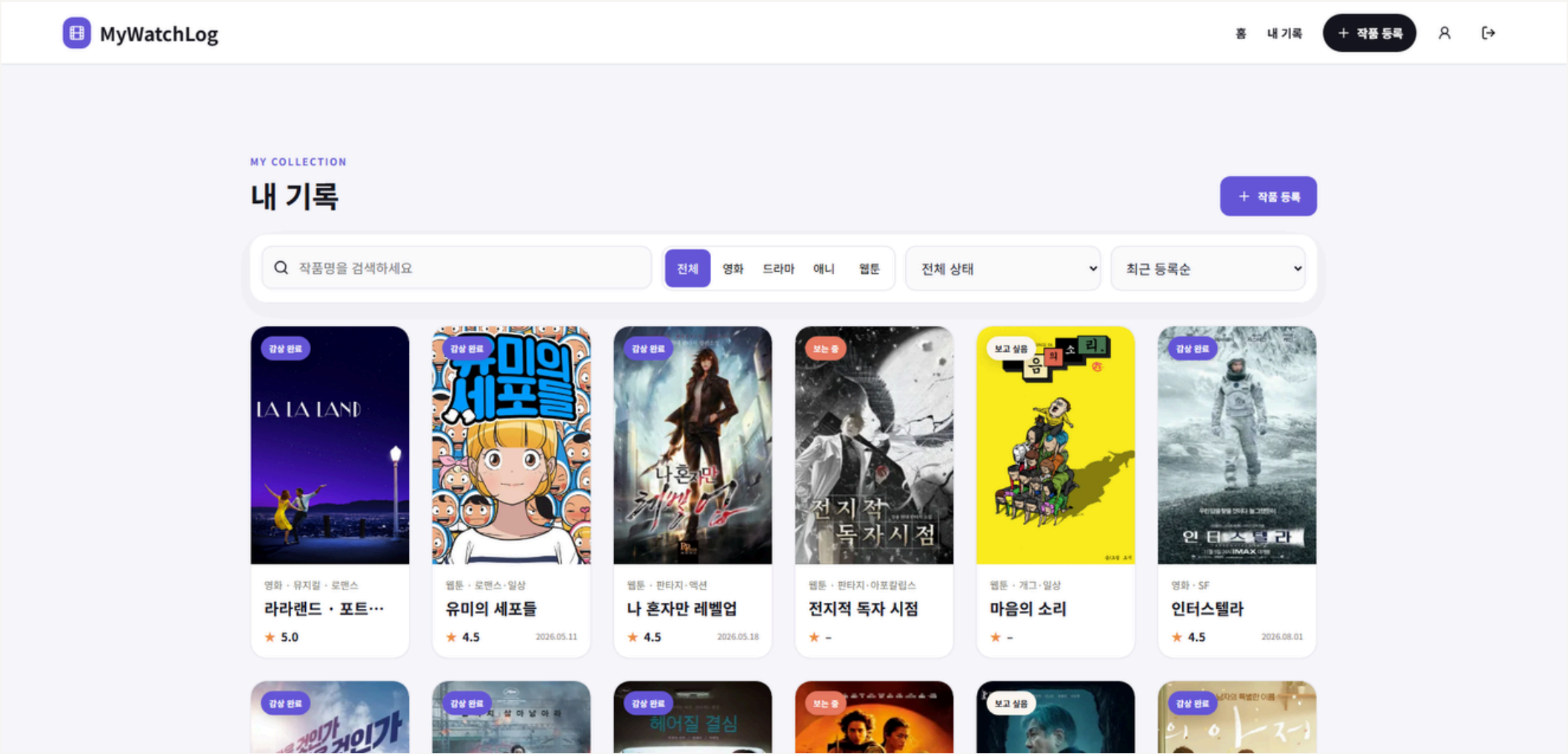
기기별로 레이아웃과 내비게이션 변경

## 주요 기능 동작 흐름



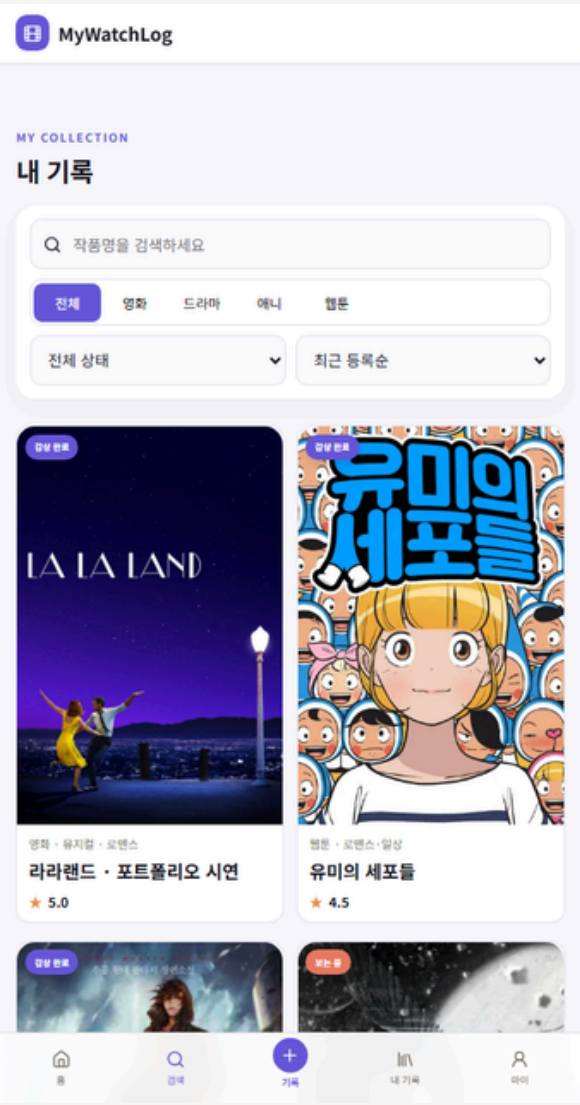
- 
- 1 사용자가 작품 정보를 입력
  - 2 프론트에서 필수 입력값 확인
  - 3 JWT와 함께 서버로 등록 요청
  - 4 서버에서 사용자와 입력값 다시 확인
  - 5 MySQL에 사용자 기록으로 저장
  - 6 저장된 기록의 상세 화면으로 이동

# 반응형 UI



DESKTOP

5열 통계 · auto-fill 포스터 그리드 · 상단 내비게이션



MOBILE

2열 그리드 · 하단 고정 내비게이션

브레이크포인트 1199px / 767px / 374px

# 개발 과정에서 해결한 문제

|    |                                                                                                                                                                                                                                                                                    |
|----|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 01 | <div>다른 사용자의 기록에 접근할 수 있는 문제</div> <div><div>CAUSE</div><div>기록 ID만으로 조회하면 해당 기록의 실제 소유자를 확인할 수 없음</div></div> <div><div>SOLUTION</div><div>기록 ID와 현재 로그인한 사용자 ID를 함께 조회하도록 변경<br/>findByIdAndUserId() 적용</div></div> <div><div>RESULT</div><div>모든 콘텐츠 API에서 계정 경계 유지</div></div> |
| 02 | <div>완료 기록의 평가 누락</div> <div><div>CAUSE</div><div>UI 검증만으로는 API 직접 호출 방어 불가</div></div> <div><div>SOLUTION</div><div>프론트 사전 검증 + 서버 비즈니스 검증</div></div> <div><div>RESULT</div><div>사용자는 빠르게 오류를 확인하고, 서버에서도 잘못된 데이터 저장을 차단</div></div>                                             |
| 03 | <div>모바일에서 데스크톱 메뉴 과밀</div> <div><div>CAUSE</div><div>작은 화면에서 헤더 액션 접근성이 낮음</div></div> <div><div>SOLUTION</div><div>767px 이하 하단 고정 메뉴와 2열 카드</div></div> <div><div>RESULT</div><div>동일 기능을 모바일 탐색 습관에 맞춤</div></div>                                                              |

## 개발하며 배운 점

- |    |                      |                                               |
|----|----------------------|-----------------------------------------------|
| 01 | 데이터 흐름               | 화면 입력부터 DB 저장, 다시 화면에 표시되는 전체 흐름을 이해          |
| 02 | 사용자 인증과<br>데이터 접근 제어 | 로그인 확인과 리소스 소유권을 매 요청마다 검증하는 방식의 차이를 체감       |
| 03 | 사용자 경험과<br>데이터 검증    | 빠른 피드백은 프론트에서, 신뢰 가능한 규칙은 서버에서 책임지는 이중 검증을 적용 |
| 04 | 반응형 설계               | 단순 축소가 아닌 내비게이션 위치와 정보 밀도를 화면 환경에 맞게 재구성      |
| 05 | AI 협업                | AI가 생성한 코드를 그대로 사용하지 않고 직접 실행·수정·검증하며 개발에 활용  |

# 기록을 저장하는 CRUD를 넘어, 개인의 기억을 안전하게 다시 찾는 흐름을 설계

---

React UI부터 JWT 인증, 사용자 소유권 검증, JPA 영속성, 반응형 탐색까지  
하나의 서비스 경험으로 연결한 1인 풀스택 프로젝트

